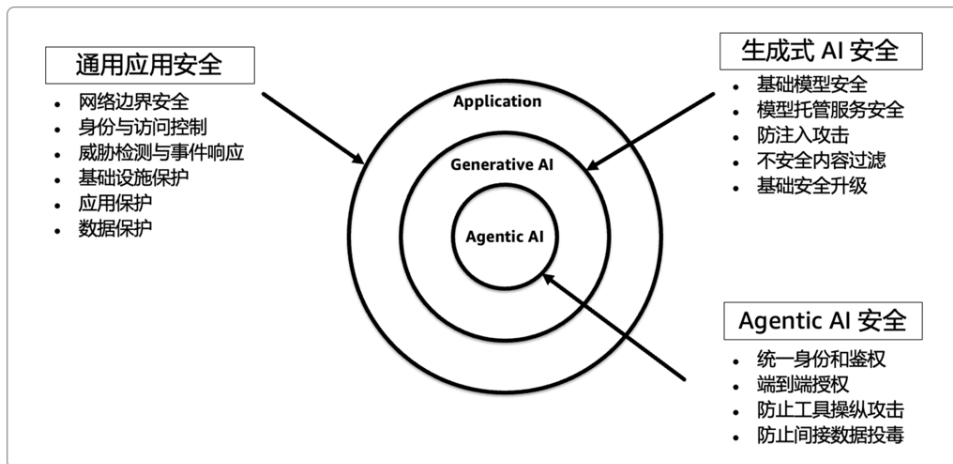


企业内部多智能体LLM系统的安全攻击与防御研究综述

引言与应用场景

企业内部部署的多智能体系统（MAS，Multi-Agent System）结合了**大语言模型（LLM）**和智能代理（Agent）技术，能够自主执行复杂任务、协同工作并与工具交互^{1 2}。这些**代理型AI**可用于**自动化运维**（如自主故障排查、DevOps流水线优化）、**信息整合与数据问答**（跨数据库检索企业知识、智能报告生成）、以及**办公协作**（数字助理安排日程、撰写文档、客户服务支持等）等场景。在银行等行业，MAS已被用于**流程自动化**：例如在KYC（了解客户）流程中，代理团队可以分工拉取公开数据、评分风险、提交监管报告，全流程无人干预但内置审计追踪和检查机制³。类似地，安全和运维团队也探索用多个智能体协同执行渗透测试、漏洞修复和监控告警等任务⁴。随着这些应用落地，企业开始构建统一的智能体开发平台，支持**多Agent协同、插件扩展、工作流编排**等功能。例如蚂蚁集团、阿里云、百度等都推出了企业级Agent平台，集成**模型支持、知识库、工作流和MCP协议**（模型上下文协议）以提升跨系统兼容性^{5 6}。

尽管MAS在企业中蕴含巨大潜能，但其**安全风险**也显著增加。多个自主智能体共享**记忆、上下文**并长时间运行，使其决策过程不透明且难以预测^{7 8}。传统针对单一LLM的安全措施在多Agent环境下已不足以覆盖扩大的攻击面⁹。现实中已出现因智能体协作失误导致的事故：如一组协同代码生成的代理因共享错误假设，彼此验证错误输出，最终产生缺陷代码¹⁰。又如AutoGPT框架中曾观测到“提示泄露”，机密信息存入持久内存后在日志或总结中意外暴露，凸显了隔离上下文和内存清理的重要性¹¹。因此，研究人员和企业开始投入对**MAS安全攻击与防御**的深入研究，希望在享受多智能体效率提升的同时，确保系统的安全可信。



为了保护企业内部的多智能体LLM系统，需要引入**多层次的安全框架**。正如下图所示，Agentic AI安全应被视为在传统应用安全和生成式AI安全之上的内层，通过身份管理、工具操控控制、内存防护等机制来抵御MAS特有威胁^{12 13}。下文我们首先分析企业MAS面临的主要攻击类型及其具体表现，然后探讨前沿的防御研究方向，并列举相关论文、框架与实践成果。

多智能体系统的攻击威胁类型

MAS引入了许多新的攻击面。基于LLM的自主代理通过语言交互和工具使用协作，但攻击者也可利用这些特性实施复杂攻击⁹。下面总结企业内部MAS可能遭遇的主要攻击类型：

- **1. 代理间通信篡改：**多Agent系统通常通过消息传递共享信息，若通信渠道缺乏保护，攻击者可**拦截、篡改或伪造代理消息**，从而误导系统行为。例如，有研究提出“中间人智能体”攻击：一个恶意代理充当通信中介，截获并修改其他代理间的信息，在消息中注入恶意逻辑^{14 15}。这类**Man-in-the-Middle**攻击会导致代理接收到伪造指令或假数据，进而执行错误决策。**企业场景下**，内部网络中未经加密的Agent通信可能被内部恶意人员窃听或篡改，造成跨部门协作Agent被误导（例如篡改财务Agent发送给风控Agent的指令）。针对该威胁，要求对代理通信进行认证和加密，以防止消息被未授权方操纵¹⁶。
- **2. 提示投毒（Prompt Injection）：**这是一种普遍的LLM攻击，在MAS中尤为危险。攻击者将**恶意指令嵌入到提供给LLM的提示或上下文中**，诱使智能体执行非预期操作¹⁷。由于多Agent常相互传递输出作为下一个Agent的输入，提示投毒可能在代理社会中**像病毒般传播**，形成“提示感染”¹⁸。例如，一个用户精心设计的输入被Agent A接收并产生携带隐藏指令的回复，Agent B读取后被欺骗执行恶意动作，如泄露敏感信息或调用危险工具¹⁸。研究表明LLM智能体架构的**提示格式静态可预测**，更易遭此攻击¹⁹。甚至出现跨Agent自我复制的注入攻击，使系统级别中断²⁰。**在企业内部**，提示注入可能通过员工提交的工单、邮件内容或知识库文档发生。如果Agent没有严格的输入校验，攻击者可在这些渠道插入恶意提示，导致AI助手泄露机密或执行未授权操作¹¹。
- **3. 身份伪造与权限升级：**在MAS中，不同Agent通常扮演不同角色，权限各异。攻击者可能**冒充某个高权限代理**或滥用身份验证漏洞，从而取得更高权限执行操作²¹。OWASP将此类威胁称为**身份欺骗与冒充**（Identity Spoofing & Impersonation）²¹。另一相关攻击是“**混淆代理**”漏洞（Confused Deputy），当Agent拥有比用户更高的权限但缺乏权限隔离时，攻击者可诱骗其代为执行自己无权的操作²²。例如，某内部Agent被授予查询数据库的权限，本应只响应合法请求，但若未校验用户输入，攻击者通过提示让Agent执行高级别查询或敏感数据修改²²。在ChatDev多Agent编程框架中，就曾观测到**角色交换攻击**：攻击者恶意更改某Agent的角色身份，使其僭越权限范围，最终导致整个Agent团队偏离预定任务²³。**企业情境下**，这可能表现为攻击者盗用某部门AI助手的令牌调用API，或者在Agent协调系统中插入伪造的“管理员Agent”发送指令。防护措施需要**严格的认证授权机制**，确保每个Agent的身份真实可信、权限边界清晰，防止未授权Agent指令进入系统²⁴。
- **4. 意图偏转与任务劫持：**攻击者利用智能体**目标规划机制**的漏洞，悄悄改变其任务意图或直接接管任务执行流程。这种**目标劫持**攻击可以是微妙的（插入次要恶意目标）或直接的（完全替换原始任务）²⁵。例如，攻击者让原本用于生成报告的Agent在输出中添加广告推广等未授权内容²⁵。Chen和Yao（2024）发现，利用LLM对**角色指令的误识别**弱点，可以诱导模型放弃原任务而去执行攻击者指定的新任务²⁶。另一种**Agent劫持**发生在工具调用链中：OWASP定义的“工具滥用”场景中，Agent接收了**对抗性操纵的数据**，结果执行了非预期操作（如调用了恶意插件）²⁷。**在企业中**，如果员工通过对话逐步诱导AI助理改变既定流程（例如本应审批报销却被引导去修改财务数据），那么AI助理等于被劫持执行了新的恶意任务。再如多个Agent协作审批流程时，攻击者诱骗任务编排代理错误地重新分配或篡改任务顺序，也是一种任务劫持行为²⁸。预防手段包括对Agent的**目标稳定性**进行校验（确保系统提示中的宗旨不被篡改）以及对任务计划引入**意图验证机制**，当任务目标突然异常变化时触发警报²⁹。
- **5. 推理操控与协同干扰：**此类攻击针对智能体更高层的**认知与协作能力**，利用其推理过程或多Agent协同机制来制造错误。攻击者可能向Agent提供**精心设计的陷阱信息**，干扰其推理链，使结论偏离正确方向³⁰。在多Agent环境下，还存在“**拜占庭智能体**”威胁：单个受损或恶意Agent通过输出虚假结果或反馈，**破坏集体决策的一致性**³¹。例如，一组协作诊断的医疗Agent中，如果一个Agent被攻击者操控报告假症状，其他Agent可能在没有外部真相源的情况下被误导，做出错误诊断决策¹⁰。另外，**级联幻觉攻击**也是一大风险：代理可能将**貌似合理但实为虚假的信息相互传播**，导致“群体思维”式的集体

错误³²。AutoGen实验就显示，代理间反复对话会放大最初的设计缺陷，所有Agent彼此强化错误结论，形成闭环谬误³³¹⁰。在企业内部，这些干扰可表现为：一个数据分析Agent给出失真的指标解释并影响了决策Agent；又或恶意Agent混入团队，在关键步骤提供错误建议让整个自动化流程偏离轨道。对此需构建**跨Agent的交叉验证和异常检测机制**，及时发现不一致或异常推理，防止单点失败演变为群体性错误¹⁰³⁴。

- **6. 敏感信息泄露**：MAS往往需要访问和处理大量内部数据，若缺乏隔离和控制，可能造成**隐私和机密的意外暴露**³⁵。持久内存与代理间频繁通信增加了泄露风险——机密信息可能在代理回复、日志记录或外部交互时泄漏出去³⁶。例如，研究发现通过提示注入可以欺骗工具型Agent**吐露其任务过程中接触的敏感个人数据**³⁷。**企业实例**包括：客服Agent将用户的私人记录错误地包含在发送给其他客户的对话中；分析Agent在总结报告时未过滤PII导致内部数据外泄到报告分享平台¹¹。还有一种情况是**访问控制不严的知识检索**：若企业将内部文档纳入LLM检索增强生成（RAG）系统，但没有严格继承源数据的权限设置，则用户可能通过提问拿到原本无权限查看的内容³⁸。同时，如果攻击者能够向企业RAG知识库写入恶意或伪造文档，也可能诱使Agent检索出**恶意植入的信息**，不仅污染回复，还可能暴露其他用户文档或敏感数据³⁹。因此，如何**精细化地管控Agent对数据的读取/写入权限**、保证上下文内敏感信息不被不当携带，是MAS安全的重要课题。
- **7. 依赖链与供应链攻击**：MAS经常通过**工具链、插件或外部API**扩展能力，这也为攻击者提供了可乘之机。一方面，如果Agent使用的插件或第三方库存在漏洞或被恶意篡改，攻击者可以利用其在Agent系统中植入后门或执行恶意代码。另一方面，**工具调用流程本身可能被滥用**：攻击者可以构造特殊输入诱导Agent调用危险系统命令或不安全函数。例如NVIDIA红队指出，若直接执行LLM生成的代码且缺乏沙箱隔离，攻击者就能通过提示注入引导模型生成恶意代码并触发远程执行（RCE），进而接管整个应用环境⁴⁰。另外，**MCP协议**（模型上下文协议）作为Agent通信和插件集成的标准，如果企业从开源社区获取未经过审计的MCP服务器实现，也会引入**供应链信任风险**。Backslash Security的报告显示已有超过7千个MCP服务器裸露在公网，许多由公共代码库部署且维护状态未知，这种无监管成长导致MCP生态存在严重安全隐患⁴¹。因此，企业在引入多Agent工具链时必须警惕**依赖项篡改、版本升级后注入**等攻击。历史上就出现过插件更新引入恶意指令、或Agent通过工具交互被植入**内存投毒**的案例⁴²。防范手段包括：严格审核引入的插件与代码、对Agent执行环境进行容器化沙箱隔离、以及对工具输出进行**格式和安全验证**（例如对Agent接收到的工具响应进行白名单过滤），最大限度降低供应链攻击面。

攻击在企业场景中的具体表现

上述攻击在企业内部的实际使用场景中可能以多种形式出现，下面通过几个典型案例说明其危害：

- **案例1：自动化IT运维中的权限滥用**。某公司部署了一套智能运维Agent团队，包含监控Agent、诊断Agent和修复Agent。监控Agent检测到服务器异常后，由诊断Agent分析故障原因，修复Agent执行重启或补丁等操作。攻击者通过在日志文件中植入**恶意提示**，使诊断Agent读取日志时触发**提示投毒**。诊断Agent于是错误地判断需要执行高危操作（删除关键配置），并指示修复Agent实施。由于缺乏**权限校验**，修复Agent具有管理员权限，直接造成系统故障停机。这体现了**提示注入和混淆代理**漏洞在企业运维中的危害：攻击者间接利用了Agent的高权限执行未经授权的操作。审计发现，如果在Agent流程中增加**输入验证和角色权限限制**（如限定修复Agent只能执行白名单内操作），可有效避免此类事故⁴³²⁴。
- **案例2：企业知识库问答中的数据泄露**。一家企业构建了内部知识问答Agent，接入了公司Wiki和文档存储作为RAG数据源。某员工在Wiki页面中存放了一些尚未公开的人事调整计划。攻击者（可能是内部知情人）故意向Agent提问相关敏感话题。由于**检索权限配置不当**，Agent在后台索引到了该保密页面内容并生成答案提供给攻击者，导致机密外泄。这一事件体现了**敏感信息泄露和访问控制缺陷**：Agent未能继承源系统对文档的权限隔离³⁸。事后改进措施包括：加强RAG数据源的**细粒度权限同步**，对Agent响

应增加**内容安全策略**（如通过Guardrail检查检索到的文档是否与查询匹配，杜绝无关敏感内容）⁴⁴。同时，对极敏感的数据集采取独立索引或更严格的审批机制，防止恶意查询导致泄密⁴⁵。

- **案例3：多Agent协同办公系统中的任务劫持。**某公司开发了智能办公助手团队，包括邮件助理Agent、日程管理Agent和报告撰写Agent，由一个中央编排Agent协调分配任务。一名内部攻击者设法获取了编排Agent的控制台访问，通过修改其配置，将自己的恶意Agent加入团队并赋予其高优先级任务。该恶意Agent伪装成“审校Agent”，实际却在所有报告中植入不当内容，并试图导出敏感数据。由于**编排系统缺少强制的Agent身份验证和信任校准**，这一**恶意内鬼Agent**成功绕过了原有安全检查，在一段时间内造成了严重的信息污染和泄露⁴⁶。这个案例综合体现了**身份伪造**（攻击者引入假Agent）、**任务劫持**（篡改任务流）、以及**协同干扰**（恶意Agent破坏整体输出）的威胁。应对措施是在多Agent编排与治理层引入**零信任模型**，每个Agent（包括新增Agent）都需经过身份审计和行为验证；同时**限制编排器权限**，对关键配置变更要求多方审核，以防单点被攻破⁴⁷³⁴。
- **案例4：客户服务多Agent系统中的内存投毒。**某企业的客服系统采用了前台客服Agent和后台策略Agent协同工作：前者与客户对话，后者根据对话记录优化客服策略。一次互动中，一位不满的用户在对话结尾使用了充满讽刺和攻击性的措辞。前台Agent虽然安抚了用户，但**将这段尖刻反馈写入共享的持久反馈缓冲区**。后台策略Agent稍后从缓冲区读取数据时，没有过滤情绪内容，错误地将讽刺语调“学到”新的回复风格，导致之后一些客户回复出现不恰当地语气。这属于**记忆投毒**（Memory Poisoning）攻击的一种：用户的恶意或不良输入污染了智能体的长期记忆/反馈循环⁴⁸⁴⁹。虽然这里用户可能无意为之，但同样的方法完全可以被恶意利用。改进措施是在Agent的记忆读写链路上增加**内容审查和情感监控**，对持久存储的反馈进行净化；并引入**反馈回路的治理机制**，例如人工审核异常反馈或定期重置Agent长期记忆，防止有害信息持续影响策略⁴⁹。

以上案例说明，在企业实际应用中，MAS的攻击往往并非单一形态，而是多种漏洞的组合利用。企业内部**封闭环境**降低了一些外部攻击面（如互联网入侵），但内部威胁和**隐蔽的提示操纵**依然存在，且由于涉及内部高敏数据和权限，其危害更甚。安全团队需要针对具体业务场景，模拟这些攻击链条进行**威胁建模和红队演练**⁵⁰，提前发现潜在薄弱环节并实施防护。

前沿防御研究方向

为应对上述安全挑战，近期的研究和实践提出了多层次、多方面的防御策略。企业和学术界探索从**架构设计、运行监控到形式化验证**的各种方法，以强化MAS的安全性⁵¹。以下分几个方面介绍前沿的防御思路：

1. Agent行为验证与审计机制

为了及时发现异常和恶意行为，MAS需要内置**行为监控和审计**组件⁴⁷。这包括对代理动作和交互进行**日志记录与分析**，以及通过机器学习模型进行**异常检测**。例如，研究者使用LSTM、自编码器模型来检测代理行为序列中的异常模式⁴⁷。微软在Copilot企业版中引入了**治理层**，实时监控多轮会话中的异常行为，确保代理响应符合法规并标记潜在有害交互⁴⁷。此外，系统可以给每个Agent维护**信任评分**，根据其历史表现动态调整对其输出的信任度，一旦某Agent的行为偏离正常范围就降低其权限或触发人工复核⁴⁷。在联邦医疗、多工厂协作等跨域场景，安全团队还采用**安全多方计算**（SMPC）等技术，让多个机构共同对智能体的诊断或检测算法出力，在不直接暴露各自数据的前提下实现**联合异常检测**⁵²。这些运行时监控与审计机制，构成了MAS安全的最后一层防线，一旦某代理失控或受攻击，可以**及时发现并阻止进一步的危害**³⁴。

2. 多Agent信任建模与访问控制

由于MAS由多角色Agent组成，必须建立**严格的信任与权限管理框架**，避免内部的“背叛”或滥用。最新研究提倡对MAS采用**零信任架构**，即默认不信任任何Agent或组件的行为，所有交互都需认证与授权⁵²。具体措施包括：为每个Agent颁发独立的**身份凭证和API令牌**，限定其可访问的数据和可调用的工具范围⁵³；在代理协作时实施**最小权限原则**，仅授予完成任务所需的最小权限，定期审计权限配置防止膨胀⁵⁴。此外，学术界开始

研究**代理间的信任度量模型和博弈论分析**。Han等人（2025）和Ko等人（2025）指出，MAS的核心漏洞在于欺骗、信任委托和串通，为此提出了形式化的跨Agent**信任验证方法**，确保即使部分Agent被攻破也不会破坏整体任务⁵⁵。还有研究将区块链或分布式日志用于MAS，记录每个Agent的关键决策，增强**责任追溯**和防篡改，杜绝事后抵赖⁵⁶。总之，通过**精细的访问控制和信任建模**，企业可以大幅减少MAS内部的横向攻击风险²⁴。这一方向在OWASP的Agent安全框架中也占据重要位置，旨在从制度和技术上防止恶意代理和内部滥权⁴⁶。

3. LLM提示保护与上下文隔离

针对提示注入等攻击，高度相关的防御思路是加强**Prompt安全**。首先，开发者应避免将**敏感指令或目标**直接硬编码在系统提示里，因为研究发现那样会显著增加Agent易被越狱或劫持的概率⁵⁷。取而代之，可以采用**多态提示和动态混淆策略**：每次会话随机改变提示的非关键部分，增加攻击者预测难度²⁹。同时，对用户输入和Agent输出的提示内容进行**过滤清洗**（如移除可疑命令语句、隐藏特定关键词），借助Guardrails等工具识别并剥离潜在恶意指令⁵⁸。另一重要技术是**上下文隔离**：将Agent的规划与执行过程隔离开来，以防止跨上下文的投毒蔓延⁴²。近期有工作将经典**规划-执行架构**重新引入LLM Agent设计，分别由不同模块处理决策和行动，彼此间只通过**结构化数据接口**交流，从而减少不受控的自由文本传播⁵⁹。Debenedetti等人（2024）的实验显示，这种模块化隔离可以使跨上下文提示注入成功率降低40%以上⁶⁰。此外，在内存管理上，可以对短期上下文和长期记忆进行**分区**，敏感信息仅在受保护的内存区域使用，Agent一般查询不到或需要额外授权才能读取。这些Prompt保护和上下文隔离机制，如**链式思想提示**、**自我审查**等，也被纳入了Agent治理框架中，以最大限度减小注入攻击面⁶¹。

4. 联合评估与异常检测方法

面对不断演化的智能体攻击，安全研究者强调需要**联合、多维度的评估体系**来衡量和提升MAS的鲁棒性⁵⁰。在学术界，已经出现多个专门用于Agent安全的**基准测试**和比赛。例如，Zhang等人（2025a）提出了包含10个场景、27类攻击的ASB基准，用于统一评估各种攻击成功率和任务完成度⁶²。Fu等人（2025c）的RAS-Eval收集了金融、医疗等领域的80个攻击场景，结果发现代理在攻击下任务完成率平均下降36.8%⁶³。这些基准有助于找出系统在何种攻击下最薄弱，并量化防御改进的效果。**异常检测**也不仅限于单代理，而是可以引入跨代理甚至跨组织的协同检测。例如前文提到的SMPC技术，可以让多个机构的Agent一起执行异常模式挖掘，又不暴露各自数据⁵²。在企业内部，不同部门的AI代理也可共享**匿名化的安全反馈**，形成**联邦式的攻击情报**：当某一代检测检测到提示注入或越权尝试时，及时通报给中央治理模块或其他代理，提高整体感知能力。微软、谷歌等正在研发自动**红队Agent**，定期模拟各类攻击以测试生产系统，从而实现持续的安全评估⁵⁰。例如，AgentDojo平台提供了97个现实任务场景，可反复验证代理在提示注入攻击下的鲁棒性⁶⁴。未来，这种自动化、多方参与的评估与检测将成为MAS安全运营的常规组成部分，帮助企业**动态发现新威胁并调整策略**⁶⁵。

5. 任务执行路径的形式化验证

为确保Agent行动序列不偏离预期且无漏洞利用，形式化方法正引起关注。一些研究团队提出对智能体的**决策逻辑和协议**进行形式化建模和验证，类似于软件模型检验，来证明某些不安全状态不可达。特别是在多Agent交互协议上，已有学者尝试使用**形式化语言验证代理通信**，以保证在各种消息序列下系统不会出现死锁或权限僭越等问题⁵²。对于关键任务（如工业控制、多无人机协作）的MAS，可以为**任务脚本和Agent计划**建立形式规约，然后用验证工具检查其满足安全性质（如“Agent X永远无法获得权限Y的资源”）。Narajala和Narayan（2025）等工作提出基于**TRiSM框架**的信任校验，通过对威胁进行分层建模，验证代理在全生命周期各阶段都遵守安全约束⁶⁶。在实践中，企业也可以利用这种方法对**Agent工作流**进行审计：例如在代码智能体场景下，定义规范“Agent不得输出未经授权的文件写操作”，然后运行Agent执行路径验证以捕捉潜在违规。形式化验证还可应用于**工具调用沙箱**本身的安全性测试⁶⁷。虽然目前形式化方法应用在LLM智能体领域还处于早期，但随着MAS在金融、医疗等高风险领域的部署，这将是确保可靠性的关键手段之一⁵²。建立**可证明安全**的Agent系统，是学术界展望的终极目标之一⁶⁸。

6. 工具/插件沙箱机制与权限最小化

多智能体系统离不开对外部工具和代码的调用。针对因此带来的RCE和数据破坏风险，业界普遍倡导为Agent配备**沙箱执行环境**，并对工具使用实施**严格的权限最小化**。NVIDIA安全团队指出，应绝对避免直接在生产环境中执行未经隔离的LLM生成代码，理想做法是将动态代码执行限制在受控沙盒中，例如基于WebAssembly的浏览器沙箱⁶⁹。通过沙箱，Agent生成的任何代码或调用的脚本只能在**隔离容器**内运行，不能访问主机系统的文件、网络或特权指令，从而杜绝了一旦Agent受骗执行恶意代码就获取整个系统权限的情况^{70 71}。同时，每个工具或插件都应配置**独立的权限**：如只允许Agent读取特定目录、调用特定API方法，而非赋予通用管理员权限。一些Agent框架（如Amazon Bedrock AgentCore Gateway）正引入**工具网关**的概念，将对外部工具的所有调用通过网关代理并实施统一的**访问控制和安全策略**⁷²。例如，在Agent调用外部数据库或REST API时，Gateway可检查其请求是否超出预定义范围、参数是否含有注入攻击特征，必要时拒绝或修改请求以保障安全⁷³。这种**安全网关+沙箱**的双重机制，被视为防范工具链攻击的最佳实践。企业应结合最小权限原则，定期审视每个Agent用到的插件和接口，取消不必要的权限，确保即便某个Agent被攻陷，攻击影响也被限制在极小范围内¹⁶。

研究进展与资源

多智能体系统安全已成为2024-2025年的研究热点，大量论文和项目涌现，涵盖攻防两端。学术界方面，**智能体安全综述**论文梳理了150余篇研究，搭建了应用、威胁、防御三大支柱的知识框架^{74 75}。其中，Wang等人AgentBench系列基准中揭示了**大型LLM智能体在提示注入攻击下的高度脆弱性**⁷⁶，Kumar等人Chiang等人分别分析了**智能体架构设计如何增加越狱攻击的风险**⁵⁷。在顶级会议和社区，MAS安全成为焦点话题：2023年OpenAI联合DEF CON举办了LLM红队挑战，许多对抗样本针对多Agent场景设计；NeurIPS等AI会议开始讨论**代理型AI**的评测标准和安全保障。知名研究团队也在积极投入：向量研究所、康奈尔大学等提出了**TRISM治理框架**，包含治理、可解释、模型运营、隐私安全四大支柱，为代理型AI提供全生命周期安全指南^{77 78}；OWASP生成式AI安全小组（ASI）发布了**Agent威胁模型和15类威胁清单**，并给出分步骤的缓解策略供企业采用^{79 46}。在工业界，微软的Guidance项目、Anthropic的安全原则、以及NVIDIA的**LLM红队实践**⁸⁰都产出了一系列最佳实践文章，为开发者提供实用建议。比如NVIDIA总结了**常见漏洞清单**（代码执行、RAG权限等）并提供了相应防护措施^{40 81}。

开源社区也涌现出多个相关工具和框架：**LangChain**集成了消息拦截和输出过滤的接口，方便开发者加入自定义安全检查；**Guardrails**开源库提供了对LLM输出进行验证的规则引擎，可用于实时阻断不合规回答；此外，研究人员开源了诸如**InjecAgent**⁸²、**AgentSmith**之类的攻防测试工具，方便为自研Agent系统进行安全体检。在多Agent安全评估上，**SafeArena**等项目可以模拟大量恶意请求测评Agent响应情况⁸³。可以预见，随着MAS在企业的全面落地，围绕其安全生态的研究与实践将更加活跃。企业应积极关注这些前沿进展，参与社区讨论与基准测试，不断**升级自身Agent系统的防护能力**。

结语

多智能体LLM系统为企业带来了前所未有的自动化和智能协作能力，但安全攻防如影随形。在内部部署场景下，MAS既面临传统IT威胁的挑战，又暴露出LLM和Agent交互所特有的新型脆弱性。通过上文梳理可以看到，攻击者能够从通信、提示、身份、推理、数据和依赖链等多个角度入手，对未加防护的Agent系统造成严重破坏。而业界也迅速响应，提出了涵盖**架构设计**（隔离、沙箱）、**运行监控**（异常检测、审计日志）、**协议治理**（零信任、形式验证）等方面的创新防御措施。安全并非一劳永逸：**攻防是一个持续拉锯的过程**⁸⁴。未来我们需要建立更完善的评估体系和动态防御机制，以适应攻击者不断翻新的技巧。在实践中，企业应采用“纵深防御”理念，将**预防、检测、响应**融入MAS生命周期，从开发阶段就考虑安全（Security by Design），并在运行中实时监控、快速响应。通过学术界、工业界的协同努力，参考最新研究成果和最佳实践，企业有望构建**安全、可信且高效**的多智能体AI系统，在实现业务创新的同时将安全风险降至最低^{9 34}。

参考文献：

1. Raza, S. 等. “代理型AI的TRiSM：LLM多智能体系统中的信任、风险和安全管理综述”，2025 85 86 .
2. 科技行者 - “安全与可信的智能代理时代：LLM多智能体系统信任、风险与安全管理综述”，2025 87 88 .
3. AWS官方博客 - “Agentic AI应用的隐私和安全”，2025 13 21 .
4. OWASP ASI - “基于Agentic AI的15类安全威胁”，2025 48 46 .
5. CSDN博客 - “智能体安全性的综述：应用、威胁与防御”，2025 89 90 .
6. NVIDIA技术博客 - “NVIDIA AI红队的实用LLM安全建议”，2025 40 81 .
7. 德勤中国 - “银行业AI智能体自动化应用洞察”，2025 3 .
8. CSDN资讯 - “2025企业级智能体开发平台报告”，2025 5 91 .
9. 安全内参 - “AI驱动的网络攻击演进：趋势与防御”，2025 92 93 .
10. 51CTO - “保护企业免受AI智能体威胁的分层防御方法”，2024 24 34 .

1 7 8 9 10 11 16 18 23 24 28 30 33 34 35 36 47 49 50 52 61 65 77 78 84 85 86 87 88 安全与可信的智能代理时代：向量研究所发布LLM多智能体系统信任、风险与安全管理全面综述 - 科技行者
<https://www.techwalker.com/2025/0607/3167384.shtml>

2 3 银行业如何借助AI智能体实现智能自动化跃迁 | 德勤中国
<https://www.deloitte.com/cn/zh/Industries/financial-services/perspectives/agentic-ai-banking.html>

4 14 15 17 19 20 25 26 29 31 37 51 55 57 60 62 63 64 66 67 68 74 75 76 82 83 89 90 92 93 智能体安全性的综述：应用、威胁与防御_智能体安全综述:应用、威胁与防御-CSDN博客
https://blog.csdn.net/hao_wujing/article/details/153481256

5 6 91 2025企业级智能体开发平台发展深入报告-CSDN.NET
<https://www.csdn.net/article/2025-08-15/150415900>

12 13 21 22 27 32 41 42 43 46 48 53 54 56 58 59 72 73 79 Agentic AI基础设施实践经验系列（八）：Agent应用的隐私和安全 | 亚马逊AWS官方博客
<https://aws.amazon.com/cn/blogs/china/privacy-and-security-of-agent-applications/>

38 39 40 44 45 69 70 71 80 81 NVIDIA AI Red 团队的实用 LLM 安全建议 - NVIDIA 技术博客
<https://developer.nvidia.cn/blog/practical-llm-security-advice-from-the-nvidia-ai-red-team/>